Chris Garlick

INTERVIEW PREPARATION

Automated Analytics

Software Engineer, Full Stack. PHP, TypeScript, data, messaging and cloud.

PREPARED FOR	PREPARED BY	DATE
Automated Analytics interview	Chris Garlick	22 October 2026

Contents

01	What they're hiring for	2
02	What they want, what to show	3
03	Incidents, problems, changes and projects	4
04	The five skills they list first	5
05	PHP and Laravel	6
06	TypeScript and asynchronous UI	7
07	SPA, PWA, components, MVP and MVVM	8
08	SQL, NoSQL and caching	10
09	Message buses and queues	11
10	REST, GraphQL and API security	12
11	CI/CD, infrastructure as code and cloud native	13
12	Git in a fast-moving team	14
13	AI, automation and analytics	15
14	Questions they're likely to ask	16
15	Questions to ask them	17
16	Preparation checklist	18

01

THE ROLE

What they’re hiring for

A remote full-stack engineer working a prioritised queue of tickets on an AI-driven marketing, recruitment and business intelligence platform.

TITLE	Software Engineer, Full Stack. Remote
THE COMPANY	AI combined with marketing, recruitment and analysis software: clients automate sales and recruitment campaigns and get business intelligence on their performance
HOW WORK ARRIVES	Tickets allocated by priority and resolution time: **incidents, problems, projects or changes**
REPORTS TO	The Head of Customer Delivery, with all allocated work, and the Chief of Product
LANGUAGES	Expert in PHP and/or TypeScript / JavaScript
FRAMEWORKS	Laravel (or CakePHP, Yii, CodeIgniter); React, Angular or Vue; Node (Express, NestJS)
DATA	MySQL, Postgres, MS SQL, NoSQL (Mongo, DynamoDB, Firestore), Redis / Memcached
MESSAGING	SQS, SNS, Pub/Sub, RabbitMQ, Kafka; Protocol Buffers and MessagePack
OPS	CI/CD, infrastructure automation, AWS or Google Cloud, serverless and containers

Read between the lines

Incidents, problems and changes are ITIL service-management terms, and you report to **Customer Delivery**. This is a client-facing engineering role: fixing what’s broken for customers quickly and calmly matters as much as building new features. The long list of technologies says *adaptable*: they want someone who picks things up fast, not an expert in all of them.

02

THE ROLE, MAPPED

What they want, what to show

Every line in the advert, with the evidence to bring. Be honest about what you've used and what you've read about.

They want	What to show
Priority-based ticket work	How you triage a queue and keep stakeholders informed.
Problem-solving	A hard bug or design problem you broke down and solved.
Communication	Explaining a technical issue to a client or non-technical colleague.
Expert PHP / TypeScript	Laravel apps you've built; typed TypeScript on the front end.
Modern front end	A SPA you built with components and async data loading.
Databases and caching	Schema design, a query you sped up, where you used Redis.
Message buses	Laravel queues, or any async job or event system you've run.
REST APIs and security	An authenticated API you designed: tokens, validation, rate limits.
CI/CD and cloud	A pipeline you built; something you deployed to AWS or similar.
Git in a fast team	Your branching, review and conflict-handling habits.

Your strongest stories

PHP / Laravel: years of it, which covers the core of the stack. **Typeset**: an API you built and run in production, with a front end, deploys and real operational problems (memory limits on a small server). **Claude Code**: you use AI tools daily, which suits a company built on AI.

03

HOW THE WORK ARRIVES

Incidents, problems, changes and projects

The advert uses ITIL language. Knowing the difference shows you understand service delivery.

Why it matters

Allocating on a priority based resolution time tickets that need to be completed. These may be: incidents, problems, projects or changes. Expect a question on how you'd prioritise.

Type	What it is	Your goal
Incident	Something is broken or degraded for a client now	Restore service fast; a workaround counts
Problem	The underlying cause of one or more incidents	Find and fix the root cause so it doesn't recur
Change	A planned change to a live system	Deliver it safely: tested, reviewed, with a rollback
Project	A larger piece of new work	Break it into deliverable pieces; report progress

Priority = impact × urgency

	High urgency	Low urgency
High impact (many clients, revenue, data)	P1: drop everything	P2: today
Low impact (one user, a workaround exists)	P3: this week	P4: when there's time

Working the queue

1

Triage first

Check new tickets for P1s before carrying on with anything else.

2

Acknowledge

Tell the reporter you have it and when they'll hear next, within the SLA.

3 **Communicate**

Update the ticket as you go. Silence is what upsets clients, not the bug.

4 **Close properly**

Confirm the fix with the reporter; link incidents to the problem behind them.

> **Say this**
> *"I'd restore service first and investigate second. Once the client is unblocked, I'd raise a problem ticket for the root cause so we fix it once rather than handling the same incident again."*

04

YOUR DUTIES

The five skills they list first

The advert lists these as *duties*, before any technology. Have one short story for each.

Skill	What they mean	Story to tell
Problem-solving	Break a complex problem down; find a new solution	A bug you couldn't reproduce at first, and how you narrowed it down
Communication	Explain technical ideas to technical and non-technical people	Explaining an outage or a trade-off to a client in plain words
Collaboration	Work well in a team	Code review you gave or received that improved the result
Adaptability	Learn new technology fast; handle changing requirements	A technology you picked up for a project (Typeset is in Rust)
Time management	Juggle priorities and meet deadlines	A week with an urgent fix landing on top of planned work

> **Structure every story the same way**
> **Situation** (one sentence), **task** (what you had to do), **action** (what *you* did, the longest part), **result** (what changed, with a number if you can). Two minutes, no more.

Explaining something technical to a non-technical person

- Start with what it means for them: *"Reports were showing yesterday's numbers."*
- Then the cause in one plain sentence: *"The job that refreshes them got stuck."*
- Then what happens next and when: *"It's fixed, and we've added an alert so we'll know first next time."*
- Leave out the jargon unless they ask for it.

05

LANGUAGES AND FRAMEWORKS

PHP and Laravel

Your strongest ground. Be ready to go deep.

Topic	What to say
PHP 8	Typed properties, constructor promotion, enums, match, readonly, named arguments, attributes.
Eloquent	Relationships; eager-load with with() to avoid N+1; scopes; casts; chunk() for big sets.
Queues and jobs	Push slow work (emails, exports, API calls) onto queues; retries, backoff, failed jobs, Horizon on Redis.
Events	Events and listeners to decouple; queue the listeners that are slow.
APIs	API resources for output, Form Requests for validation, Sanctum for tokens, rate limiting middleware.
Testing	Feature tests with RefreshDatabase; fakes for Queue, Mail, Http; Pest or PHPUnit.
Other frameworks	CakePHP, Yii and CodeIgniter share the same MVC ideas: routes, controllers, models, views.

```
// Pull campaign stats from a third-party API, safely
final class SyncCampaignStats implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable;

    public int $tries = 5;
    public array $backoff = [10, 60, 300];

    public function __construct(
        public readonly Campaign $campaign,
    ) {}

    public function handle(AdsClient $ads): void
    {
        $stats = $ads->statsFor($this->campaign->external_id);

        // Idempotent: a second run updates the same row
        CampaignStat::updateOrCreate(
            ['campaign_id' => $this->campaign->id, 'date' => $stats->date],
            ['impressions' => $stats->impressions, 'clicks' => $stats->clicks],
        );
    }
}
```

06

LANGUAGES AND FRAMEWORKS

TypeScript and asynchronous UI

The other half of *expert in PHP or TypeScript*, and the *asynchronous UI development patterns* they list.

Topic	What to say
Types	Interfaces vs type aliases; unions and narrowing; generics; strict mode on.
Utility types	Partial, Pick, Omit, Record; unknown instead of any for untrusted data.
Promises	async / await; Promise.all for parallel work, allSettled when some may fail.
Event loop	One thread; awaiting yields so the UI stays responsive.
Race conditions	A slow old request overwriting a newer one: cancel with AbortController.
Debounce	Wait until typing stops before searching, so you don’t fire a request per key.

Topic	What to say
UI states	Every async view has four: loading, error, empty and success.
Optimistic updates	Show the change at once, roll back if the server rejects it.

```ts
type Result<T> =
  | { ok: true; data: T }
  | { ok: false; error: string };

let ctrl: AbortController | null = null;

export async function search(
  q: string,
): Promise<Result<Candidate[]>> {
  ctrl?.abort();          // cancel the last search
  ctrl = new AbortController();
  const url = `/api/candidates?q=${encodeURIComponent(q)}`;
  try {
    const res = await fetch(url, { signal: ctrl.signal });
    if (!res.ok) {
      return { ok: false, error: `HTTP ${res.status}` };
    }
    return { ok: true, data: await res.json() };
  } catch (e) {
    if (e instanceof DOMException
        && e.name === 'AbortError') {
      return { ok: false, error: 'cancelled' };
    }
    throw e;
  }
}
```

07

MODERN APPLICATION DEVELOPMENT

SPA, PWA, components, MVP and MVVM

They want a *desire to stay current*. Be able to define each term and say when you'd use it.

Term	What it is	When to use it
SPA	One page that loads once; JavaScript renders views and fetches data	App-like dashboards, like a BI or campaign tool
PWA	A web app with a manifest and a service worker: installable, works offline, can push notifications	Users on mobile or patchy connections; no app store needed

Term	What it is	When to use it
Components	UI built from small, reusable pieces with props in and events out	Always; it's how React, Vue and Angular work
MVP	Model, View, Presenter: a passive view; the presenter handles logic and updates the view	Testable UI logic with a thin view
MVVM	Model, View, ViewModel: the view binds to the view model's state and updates automatically	Data-binding frameworks: Vue and Angular are close to MVVM

SPA TRADE-OFFS	Fast after the first load, but a slower first paint and harder SEO; server rendering (Nuxt, Next) or Inertia helps.
SERVICE WORKER CACHING	Cache first for static assets; network first for API data so it's never stale.
STATE	Keep state local until it's shared; then a store (Pinia, Redux, Zustand). Server data in a query cache (TanStack Query).
LARAVEL + SPA	Inertia.js gives a SPA feel with Laravel routing and no separate API.

```
// Vue 3: the component is the view model;
// the template binds to its state.
<script setup lang="ts">
import { ref, computed, onMounted } from 'vue'

const props = defineProps<{ campaignId: number }>()
const stats = ref<Stat[]>([])
const loading = ref(true)
const ctr = computed(() => {
  const sum = (k: 'impressions' | 'clicks') =>
    stats.value.reduce((n, s) => n + s[k], 0)
  const imps = sum('impressions')
  const clicks = sum('clicks')
  return imps ? (clicks / imps) * 100 : 0
})

onMounted(async () => {
  stats.value = await api.stats(props.campaignId)
  loading.value = false
})
</script>
```

08

DATA

SQL, NoSQL and caching

They list almost every kind of database. Show you know which to choose and why.

Store	Strength	Watch out for
MySQL / Postgres	Relational data, joins, transactions, reporting	Indexes, slow queries on big tables
MS SQL	The same ideas in the Microsoft world; T-SQL	Syntax differences: TOP, not LIMIT
MongoDB	Flexible documents; nested data read together	No joins by default; design for your queries
DynamoDB	Huge scale, single-digit millisecond reads by key	Design partition keys around access patterns up front
Firestore	Real-time sync to web and mobile clients	Query limits; cost per read
Redis	Cache, queues, sessions, counters, rate limits, pub/sub	It's in memory: set TTLs, plan for eviction
Memcached	A simple, fast key-value cache	No persistence and no data structures

Caching patterns

CACHE-ASIDE	Read the cache; on a miss, read the database and store the result with a TTL. Laravel's Cache::remember does this.
INVALIDATION	Delete or update the key when the data changes; use tags or versioned keys for groups.
STAMPEDE	Many requests miss at once and all hit the database; use a lock so only one rebuilds the value.

```sql
-- Click-through rate by campaign, last 7 days
SELECT c.name,
       SUM(s.impressions) AS impressions,
       SUM(s.clicks)      AS clicks,
       ROUND(100 * SUM(s.clicks)
             / NULLIF(SUM(s.impressions), 0), 2) AS ctr
FROM campaigns c
JOIN campaign_stats s ON s.campaign_id = c.id
WHERE s.date >= CURRENT_DATE - INTERVAL 7 DAY
GROUP BY c.id, c.name
ORDER BY ctr DESC;
```

09

DATA STREAMING

Message buses and queues

SQS, SNS, Pub/Sub, RabbitMQ and Kafka. Know the differences and the rules that apply to all of them.

Why it matters

An automation and analytics platform runs on background work: campaign sends, data imports, report builds. Messaging is how that work is spread out and kept reliable.

Tool	Model	Use it for
SQS	A queue: each message is handled by one consumer	Background jobs on AWS; Laravel's SQS driver
SNS	Publish / subscribe: one message fans out to many subscribers	SNS → several SQS queues, each doing its own job
Google Pub/Sub	Pub/sub with push or pull subscriptions	The same on Google Cloud
RabbitMQ	A broker with exchanges routing to queues	Flexible routing, priorities, work queues
Kafka	A durable, ordered log, split into partitions; consumers replay it	High-volume event streams and analytics pipelines

AT-LEAST-ONCE	Most systems may deliver a message twice, so consumers must be **idempotent**: processing twice has the same result as once.
DEAD-LETTER QUEUE	Messages that keep failing go aside for inspection instead of blocking the queue.
ORDERING	SQS standard queues don't guarantee order; SQS FIFO and Kafka (within a partition) do.
VISIBILITY TIMEOUT	In SQS, a message hides while being processed; if the worker dies, it reappears.
BACKPRESSURE	Watch queue depth and message age; scale workers on them.

Protocol Buffers and MessagePack

Format	What it is	Why use it
JSON	Text, self-describing	Readable, universal; but larger and slower to parse
MessagePack	Binary JSON: same data model, no schema	Smaller and faster, drop-in for JSON
Protocol Buffers	Binary with a schema in .proto files; code generated per language	Compact, typed, versioned; used by gRPC

> **Protobuf schema evolution**
> Never reuse or renumber a field tag; add new fields as optional; mark removed ones `reserved`. That keeps old and new services able to read each other's messages.

10

APIS

REST, GraphQL and API security

Designing and building RESTful APIs, loose familiarity with GraphQL, and API security best practice.

REST principle	In practice
Resources and verbs	GET /campaigns, POST /campaigns, PATCH /campaigns/42, DELETE /campaigns/42
Status codes	200, 201 Created, 204, 400, 401 vs 403, 404, 409, 422 validation, 429, 500
Pagination	Cursor-based for large or changing lists; limit the page size
Versioning	/v1/ in the path or a header; never break existing clients
Idempotency	PUT and DELETE are idempotent; an Idempotency-Key header makes POST safe to retry
Consistency	One error format, one date format (ISO 8601), consistent naming

GRAPHQL

One endpoint; the client asks for exactly the fields it needs. Good for complex front ends; harder to cache. Watch for N+1 queries (use DataLoader) and limit query depth.

API security

Risk	Defence
Broken object-level authorisation	The top API risk: check the user may see *this* record, not just that they're logged in. Laravel policies.
Broken authentication	Short-lived tokens, OAuth 2 for third parties, hashed API keys, HTTPS only.
Mass assignment	Validate and whitelist fields; $fillable in Laravel.
Injection	Parameterised queries; validate all input.
Abuse	Rate limiting per user and per key; size limits on requests.
Data exposure	Return only the fields needed (API resources); never leak stack traces.
Multi-tenancy	Every query scoped to the client's account; test that one client can't see another's data.

11

DEVOPS

CI/CD, infrastructure as code and cloud native

Familiarity and *understanding* are the words they use: you don't need to be an ops engineer.

A good pipeline

1

On every push

Install (cached), lint, static analysis (PHPStan, tsc), unit and feature tests.

2

On merge

Build once: a Docker image or bundle. Deploy it to staging and run smoke tests.

3

To production

Promote the same artefact; migrations first; health check; easy rollback.

Infrastructure tool	What it is
Terraform	Declares infrastructure in code for any cloud; plan, then apply
CloudFormation / CDK	AWS's own; CDK lets you write it in TypeScript
Ansible	Configures servers by running tasks over SSH

Infrastructure tool	What it is
Docker	Packages the app and its dependencies so it runs the same everywhere

Cloud native

Service	What it runs	Good for
AWS Lambda / Cloud Functions	A function per event; pay per run	Webhooks, file processing, scheduled jobs, queue consumers
AWS App Runner / Cloud Run	A container, scaled automatically, even to zero	Web apps and APIs without managing servers
App Engine	Your code on Google's managed platform	Simple web apps

> **Serverless trade-offs**
> Cold starts, time limits (15 minutes on Lambda), and database connections (use RDS Proxy). For PHP on Lambda, **Bref** is the usual route; Laravel Vapor is built on it.

12

VERSION CONTROL

Git in a fast-moving team

They ask for *expertise*. Show habits, not just commands.

Practice	Why
Short-lived branches	Merge within a day or two; fewer, smaller conflicts.
Trunk-based or GitHub flow	main is always deployable; feature flags for unfinished work.
Small PRs	Faster, better reviews; easier to revert.
Good commit messages	Say *why* the diff already says what.
Rebase your branch, don't rewrite shared history	Rebase onto main before merging; never force-push a shared branch.
Revert, don't reset, in production	git revert makes a new commit that undoes a bad one, safely.

```
git switch -c fix/ctr-divide-by-zero
git fetch origin && git rebase origin/main  # stay current
git add -p                    # stage only what belongs
git commit -m "Don't divide by zero with no impressions"
git push --force-with-lease   # safe after a rebase

# Find the commit that broke it
git bisect start <bad> <good>
# Undo a bad commit that's already in main
git revert <sha>
# When did this code change?
git log -S "ctr" --oneline
```

13

THE PRODUCT

AI, automation and analytics

Show interest in what they actually build: campaign automation and business intelligence for sales and recruitment.

CAMPAIGN AUTOMATION	Scheduled and triggered sends across channels, at volume: queues, retries, rate limits from providers, and unsubscribes respected.
INTEGRATIONS	Ad platforms, CRMs, job boards and ATS systems: OAuth, webhooks, API limits, and data that arrives late or twice.
ANALYTICS PIPELINE	Events collected, cleaned and aggregated into daily tables, so dashboards read summaries, not raw events.
ACCURACY	“Accurate and fast” is their promise: timezones, duplicates and late data are where numbers go wrong.
AI FEATURES	Likely scoring, summarising or generating content: structured output, cost and latency, and checking quality.
DATA PROTECTION	Candidate and customer data is personal data under UK GDPR: access control, retention and consent.

A good question to ask yourself before the interview

Look at their website and pick one feature. How would you build it? What would break at 10× the volume? Being able to say that in the interview shows real interest.

Say this

"What I like about this role is the mix: client-facing tickets where you see the impact straight away, plus the data and automation side, which is where AI is most useful."

14

PREPARATION

Questions they're likely to ask

Technical

- Three tickets arrive at once: an outage for one client, a bug for many, and a change due today. What order, and why?
- What's the difference between an incident and a problem?
- How would you stop a Laravel page making hundreds of database queries?
- Explain how you'd handle a search box that fires a request on every keystroke.
- What's the difference between a SPA and a PWA? Between MVP and MVVM?
- When would you choose DynamoDB or Mongo over MySQL?
- SQS and Kafka: what's the difference, and when would you use each?
- A message might be delivered twice. How do you make that safe?
- How would you secure a REST API used by many client companies?
- Describe a CI/CD pipeline you'd set up for a Laravel and Vue app.
- How do you undo a bad commit that's already in main?

About you

- Tell me about a complex problem you solved.
- Explain something technical you worked on as if I weren't a developer.
- Tell me about a technology you had to learn quickly.
- How do you manage competing priorities and deadlines?
- Why Automated Analytics?

15

PREPARATION

Questions to ask them

Pick three or four that fit the conversation.

The work

- What's the split between incidents and changes versus project work, week to week?
- How are priorities and resolution times set, and who decides when they conflict?
- How does working with both Customer Delivery and the Chief of Product work in practice?

The technology

- What does the stack look like today: which of the technologies in the advert are in use?
- Where does AI sit in the product, and what's next for it?
- What does the path from commit to production look like? How often do you deploy?
- What's the biggest technical challenge the platform has right now?

The team

- How big is the engineering team, and how do you work together remotely?
- What would success look like in the first three and six months?
- How do you support people learning new technologies?

Closing line

"I'm a strong PHP and Laravel developer who's comfortable across the stack and picks up new tools quickly. I like owning a ticket from the client's report to the fix in production, and keeping people informed along the way. I think I'd fit well here."

16

BEFORE THE INTERVIEW

Preparation checklist

The night before

1

Look them up

Automated Analytics' product, clients and recent news; who's interviewing you.

2

Rehearse five stories out loud

One for each skill they list: problem-solving, communication, collaboration, adaptability, time management.

3

Know your gaps

Which listed technologies you've used, and which you've only read about. Say so honestly, then say how you'd learn.

4

Practise the definitions

Incident vs problem, SPA vs PWA, MVP vs MVVM, SQS vs Kafka, out loud.

On the day

5

Re-read the advert

Its words are your vocabulary: *priority, resolution time, incidents, problems, changes*.

6

Check your setup

Camera, sound, a working editor and terminal in case of a live exercise.

7

Have this guide open

Plus your questions for them, and a glass of water.